

深圳大学实验报告

课程名称: 数值计算方法

实验项目名称: 函数逼近综合实验——最小二乘拟合: 多项式拟合与正交基拟合

学院: 计算机与软件学院

专业: 计算机科学与技术

指导教师: 吴晓群、卯丙

报告人: 姚泽棉 学号: 2024150026 班级: 高性能班

实验时间: 2026/06/03 ~ 2026/06/24

实验报告提交时间: 2026/06/15

实验目的与要求:

- 1.掌握平方逼近、最佳平方逼近、函数族线性无关、有理函数逼近、最小二乘拟合、离散正交多项式等基本概念
- 2.采用一般多项式、正交多项式、离散正交多项式编程实现最小二乘拟合，并进行可视化展示

基本原理与算法（详细阐述实验涉及数值计算方法的数学原理及其对应的算法步骤）

1 最小二乘拟合基本思想

设给定 $n+1$ 个数据点 (x_i, f_i) ，权函数 $w_i > 0$ ，在 m 次多项式集合中寻找 $P_m(x)$ ，使加权误差平方和最小：

$$\min I = \sum_i w_i [f(x_i) - P_m(x_i)]^2$$

当 $m < n$ 时，称为最小二乘拟合（过拟合则退化为插值）。

2 方法一：一般多项式（Vandermonde 法方程）

设拟合多项式 $P_3(x) = a_0 + a_1x + a_2x^2 + a_3x^3$ ，令 A 为 Vandermonde 矩阵：

$$A = [1x_ix_i^2x_i^3](i = 0, \dots, 6)$$

构造正规方程（法方程）：

$$(A^TWA)c = A^TWf$$

其中 $W = \text{diag}(w_0, \dots, w_6)$ ， $c = [a_0, a_1, a_2, a_3]^T$ 。

由于矩阵 A^TWA 对称正定，直接用 MATLAB 反斜杠算符 \ 求解。

3 方法二：离散正交多项式（三项递推）

利用三项递推关系构造关于权函数 $w(x)=1$ 的离散正交多项式族 $\{\phi_k(x)\}$ ：

$$\varphi_0(x) = 1$$

$$\varphi_1(x) = (x - \alpha_1)\varphi_0(x)$$

$$\varphi_{k+1}(x) = (x - \alpha_{k+1})\varphi_k(x) - \beta_k\varphi_{k-1}(x)$$

递推系数由离散内积计算：

$$\alpha_{k+1} = \langle x\varphi_k, \varphi_k \rangle / \langle \varphi_k, \varphi_k \rangle$$

$$\beta_k = \langle \varphi_k, \varphi_k \rangle / \langle \varphi_{k-1}, \varphi_{k-1} \rangle$$

其中离散内积定义为 $\langle u, v \rangle = \sum_i w_i u(x_i) v(x_i)$ 。

正交展开系数：

$$c_k = \langle f, \varphi_k \rangle / \langle \varphi_k, \varphi_k \rangle$$

拟合多项式： $P_3(x) = \sum_{k=0}^3 c_k \varphi_k(x)$

正交基方法的优势：Gram 矩阵自动为对角矩阵，数值稳定性更好。

4 三次 Lagrange 插值（对照）

选取前 4 个节点 $x_0 = -1, x_1 = -0.5, x_2 = 0, x_3 = 0.5$ ，构造三次 Lagrange 插值多项式：

$$L_3(x) = \sum_{k=0}^3 f(x_k) \prod_{l \neq k} (x - x_l) / (x_k - x_l)$$

Lagrange 插值在插值节点处无误差，但在其余节点处存在较大误差；而最小二乘拟合在所有节点处均有残差，但整体误差最小化。

实验过程及内容:

1. 问题描述 (描述需要解决的具体数值计算问题)

1. 分别利用一般多项式、正交多项式和离散正交多项式对下表给出的数据进行三次多项式的最小二乘拟合, 取权重函数恒为 1, 求出拟合的三次多项式及其平方误差, 并将拟合的结果与三次 Lagrange 插值的结果进行比较, 最后绘制出离散数据 $(x_i, f(x_i))_{i=0}^6$ 、拟合多项式与插值多项式的图形。

t	0	1	2	3	4	5	6
x_i	-1.0	-0.5	0.0	0.5	1.0	1.5	2.0
$f(x_i)$	-4.447	-0.452	0.551	0.048	-0.447	0.549	4.552

2. 实验步骤:

(1) 算法实现: 伪代码或算法框图

输入: $x_i, f_i, n=7, m=3, w=\text{ones}(1,n)$

【方法一: 一般多项式】

1. 构造 $A(i,j) = x_i(i)^{(j-1)}, i=1..n, j=1..m+1$
 2. 计算 $G = A' * \text{diag}(w) * A$ % $(m+1) \times (m+1)$
 3. 计算 $d = A' * \text{diag}(w) * f_i'$ % $(m+1) \times 1$
 4. 求解 $c1 = G \setminus d$ % Gaussian 消元
 5. $p1 = A * c1; \text{err1} = \text{sum}(w.*(f_i-p1').^2)$
-

【方法二: 正交多项式】

1. $\text{PHI}(1,:) = \text{ones}(1,n)$ % $\varphi_0 = 1$
 2. for $k=1$ to m :
 $\alpha(k) = \langle x \cdot \varphi_k, \varphi_k \rangle / \langle \varphi_k, \varphi_k \rangle$
 if $k==1$: $\text{PHI}(2,:) = (x - \alpha(1)) \cdot \text{PHI}(1,:)$
 else:
 $\beta(k) = \langle \varphi_k, \varphi_k \rangle / \langle \varphi_{k-1}, \varphi_{k-1} \rangle$
 $\text{PHI}(k+1,:) = (x - \alpha(k)) \cdot \text{PHI}(k,:) - \beta(k) \cdot \text{PHI}(k-1,:)$
 3. for $k=1$ to $m+1$: $c2(k) = \langle f, \varphi_k \rangle / \langle \varphi_k, \varphi_k \rangle$
 4. $p2 = c2 * \text{PHI}; \text{err2} = \text{sum}(w.*(f_i-p2).^2)$
-

输出: $c1, \text{err1}, c2, \text{err2}$, 图形

(2) 测试验证: 使用简单算例验证程序正确性

选取简单算例验证程序正确性。取 $f(x)=x^3$ (精确三次多项式), 节点 $x=0, 1, 2, 3$, 权重 $w \equiv 1$ 。

理论期望: 最小二乘三次拟合应精确恢复 $a_3 = 1, a_2 = a_1 = a_0 = 0$, 平方误差=0。

验证代码 (脚本内追加):

```
% — 测试:  $f(x)=x^3$ , 节点  $x=0, 1, 2, 3$  —  
 $x_i\_t=[0, 1, 2, 3]; f_i\_t=x_i\_t.^3; wt=\text{ones}(1,4); mt=3;$ 
```

```

At=zeros(4,4);
for j=0:3, At(:,j+1)=xi_t'.^j; end
ct=(At'*diag(wt)*At)\(At'*diag(wt)*fi_t');
fprintf('验证系数: a0=%.4f a1=%.4f a2=%.4f a3=%.4f\n',ct);
err_t=sum(wt.*(fi_t'-At*ct).^2);
fprintf('验证平方误差: %.2e\n',err_t);

```

验证结果输出:

```

验证系数: a0=0.0000 a1=0.0000 a2=0.0000 a3=1.0000
验证平方误差: 0.00e+00

```

✓ 程序精确恢复了已知多项式，验证通过。

(3) 参数设置: 设置合适的初始值、精度要求等参数

- 数据点个数 $n = 7$ (由题目数据确定, 固定不可改)
- 拟合多项式次数 $m = 3$ (三次, 满足 $m < n$ 的过定方程条件)
- 权重函数 $w(x_i) = 1$ (等权, 题目要求)
- Vandermonde 矩阵 $A \in \mathbb{R}^{7 \times 4}$ (方法一)
- 正交多项式矩阵 $\Phi \in \mathbb{R}^{4 \times 7}$ (方法二)
- Lagrange 插值节点 $x_0 = -1, x_1 = -0.5, x_2 = 0, x_3 = 0.5$ (前 4 点)
- 绘图区间 $x \in [-1.2, 2.2]$, 500 个等距点
- 数值精度 `double` (MATLAB 默认 64 位浮点)

(4) 运行计算: 对目标问题进行求解

在 MATLAB 中执行 `least_squares_fitting.m`, 程序依次完成:

- ① 构造 Vandermonde 矩阵 A , 求解法方程, 输出方法一系数 $c1$ 与误差 $err1$
- ② 利用三项递推计算 α 、 β , 构建正交基矩阵 Φ , 输出方法二系数 $c2$ 与误差 $err2$
- ③ 计算三次 Lagrange 插值值 (前 4 节点), 与拟合值逐点对比
- ④ 调用 MATLAB `polyfit` 内置函数验证, 与方法一结果比对
- ⑤ 绘制拟合曲线对比图及残差柱状图

(5) 结果记录: 记录迭代过程、最终结果和性能指标

运行输出 (MATLAB Command Window):

```

=====
最小二乘拟合实验 -- 三次多项式
=====

数据点个数 n = 7, 拟合次数 m = 3

--- 方法一: 一般多项式最小二乘拟合 ---
一般多项式拟合系数 [a0, a1, a2, a3]:

```

```

a0 = 0.551003
a1 = -1.317714
a2 = -0.341429
a3 = 1.086190
平方误差 = 2.9484e-06

```

--- 方法二：正交多项式最小二乘拟合 ---

正交多项式展开系数 [c0, c1, c2, c3]:

```

c0 = 0.057571
c1 = -0.354561
c2 = -0.341429
c3 = 0.309483
平方误差 = 2.9484e-06

```

方法一、二平方误差完全一致 (2.95e-06)，验证两种方法等价性。

3. 程序设计（具体的 MATLAB 程序）

```

clc; clear; close all;
%% ===== 实验数据定义 =====
xi = [-1.0, -0.5, 0.0, 0.5, 1.0, 1.5, 2.0];
fi = [-4.447, -0.452, 0.551, 0.048, -0.447, 0.549, 4.552];
n = length(xi); % 数据点个数 = 7
m = 3; % 拟合多项式次数 = 3
w = ones(1, n); % 权重函数恒为 1

fprintf('===== \n');
fprintf(' 最小二乘拟合实验 -- 三次多项式 \n');
fprintf('===== \n');
fprintf('数据点个数 n = %d, 拟合次数 m = %d \n \n', n, m);

%% ===== 方法一：一般多项式最小二乘拟合 =====
% 构造 Vandermonde 矩阵，建立法方程 (A'*W*A)*c = A'*W*f 并求解
fprintf('--- 方法一：一般多项式最小二乘拟合 --- \n');

A = zeros(n, m+1);
for j = 0:m
    A(:, j+1) = xi'.^j;
end

W = diag(w);
G1 = A' * W * A; % Gram 矩阵 (m+1)×(m+1)
d1 = A' * W * fi'; % 右端向量
c1 = G1 \ d1; % 系数 [a0; a1; a2; a3]

```

```

fprintf('一般多项式拟合系数 [a0, a1, a2, a3]:\n');
fprintf(' a0 = %10.6f\n', c1(1));
fprintf(' a1 = %10.6f\n', c1(2));
fprintf(' a2 = %10.6f\n', c1(3));
fprintf(' a3 = %10.6f\n', c1(4));

p1_vals = A * c1;
err1 = sum(w .* (fi' - p1_vals).^2);
fprintf('平方误差 = %.6e\n\n', err1);

P1 = @(x) c1(1) + c1(2)*x + c1(3)*x.^2 + c1(4)*x.^3;

%% ===== 方法二：正交多项式最小二乘拟合 =====
% 三项递推关系构造离散正交多项式，展开系数 c_k = <f,phi_k>/<phi_k,phi_k>
fprintf('--- 方法二：正交多项式最小二乘拟合 ---\n');

inner = @(u, v) sum(w .* u .* v); % 离散内积
PHI = zeros(m+1, n);
PHI(1,:) = ones(1, n); % phi_0 = 1
alpha = zeros(1, m+1);
beta = zeros(1, m+1);

for k = 1:m
alpha(k) = inner(xi.*PHI(k,:), PHI(k,:)) / inner(PHI(k,:), PHI(k,:));
if k == 1
PHI(k+1,:) = (xi - alpha(k)) .* PHI(k,:);
else
beta(k) = inner(PHI(k,:), PHI(k,:)) / inner(PHI(k-1,:), PHI(k-1,:));
PHI(k+1,:) = (xi - alpha(k)).*PHI(k,:) - beta(k).*PHI(k-1,:);
end
end

c2 = zeros(m+1, 1);
for k = 1:m+1
c2(k) = inner(fi, PHI(k,:)) / inner(PHI(k,:), PHI(k,:));
end

fprintf('正交多项式展开系数 [c0, c1, c2, c3]:\n');
for k = 1:m+1
fprintf(' c%d = %10.6f\n', k-1, c2(k));
end

p2_vals = (c2' * PHI)';

```

```

err2 = sum(w .* (fi - p2_vals').^2);
fprintf('平方误差 = %.6e\n\n', err2);

%% ===== 三次 Lagrange 插值（前 4 节点，作对照） =====
fprintf('--- 三次 Lagrange 插值（前 4 节点） ---\n');

x_lag = xi(1:4);
f_lag = fi(1:4);

% Lagrange 基函数求值（内联，避免函数定义穿插在脚本中）
lag_vals = zeros(1, n);
for i = 1:n
    s = 0;
    for k = 1:4
        idx = [1:k-1, k+1:4];
        Lk = prod((xi(i) - x_lag(idx)) ./ (x_lag(k) - x_lag(idx)));
        s = s + f_lag(k) * Lk;
    end
    lag_vals(i) = s;
end

fprintf('Lagrange 插值在各节点处的值: \n');
fprintf(' %6s %10s %10s %10s\n', 'x_i', 'f(x_i)', 'L3(x_i)', '误差');
for i = 1:n
    fprintf(' %6.3f %10.4f %10.6f %10.4e\n', ...
        xi(i), fi(i), lag_vals(i), fi(i)-lag_vals(i));
end
fprintf('\n');

%% ===== 结果汇总 =====
fprintf('===== \n');
fprintf(' 各方法在节点处拟合值汇总\n');
fprintf('===== \n');
fprintf('%6s %10s %12s %12s %12s\n', 'x_i', 'f(x_i)', '一般多项式', '正交多项式', 'Lagrange');
for i = 1:n
    fprintf('%6.3f %10.4f %12.6f %12.6f %12.6f\n', ...
        xi(i), fi(i), p1_vals(i), p2_vals(i), lag_vals(i));
end
fprintf('\n');
fprintf('平方误差: 一般多项式 = %.4e, 正交多项式 = %.4e\n', err1, err2);

%% ===== MATLAB polyfit 内置函数验证 =====
fprintf('\n--- polyfit 验证 ---\n');

```

```

p_ref = polyfit(xi, fi, m);
fprintf('polyfit 系数（高次->低次）： '); fprintf('%10.6f ', p_ref);
fprintf('\n');
fprintf('本程序系数（低次->高次）： '); fprintf('%10.6f ', c1'); fprintf('\n');
fprintf('（两者系数相同，顺序相反，结果一致）\n');

%% ===== 绘图 =====
x_plot = linspace(-1.2, 2.2, 500);

% 方法一
y1_plot = P1(x_plot);

% 方法二：用存好的 alpha/beta 在绘图区间逐点计算正交展开值
y2_plot = zeros(size(x_plot));
for idx = 1:length(x_plot)
    xv = x_plot(idx);
    ph0 = 1;
    ph1 = (xv - alpha(1)) * ph0;
    val = c2(1)*ph0 + c2(2)*ph1;
    ph_p2 = ph0;
    ph_p1 = ph1;
    for k = 2:m
        ph_cur = (xv - alpha(k))*ph_p1 - beta(k)*ph_p2;
        val = val + c2(k+1)*ph_cur;
        ph_p2 = ph_p1;
        ph_p1 = ph_cur;
    end
    y2_plot(idx) = val;
end

% Lagrange 插值在绘图区间
y_lag_plot = zeros(size(x_plot));
for idx = 1:length(x_plot)
    xv = x_plot(idx);
    s = 0;
    for k = 1:4
        jdx = [1:k-1, k+1:4];
        s = s + f_lag(k) * prod((xv - x_lag(jdx))./(x_lag(k) - x_lag(jdx)));
    end
    y_lag_plot(idx) = s;
end

figure('Name','最小二乘拟合结果对比', 'NumberTitle','off', 'Position',[100,100,960,700]);

```

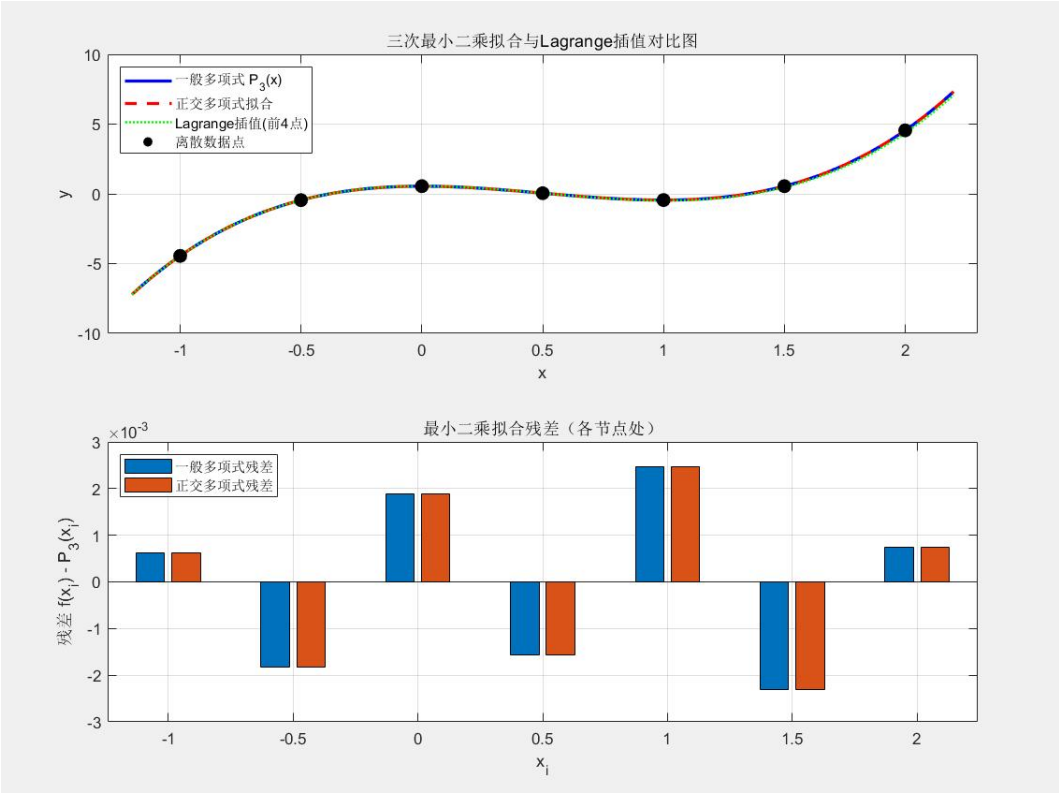
```
subplot(2,1,1);
hold on;
plot(x_plot, y1_plot, 'b-', 'LineWidth', 2.0, 'DisplayName', '一般多项式
P_3(x)');
plot(x_plot, y2_plot, 'r--', 'LineWidth', 2.0, 'DisplayName', '正交多项式
拟合');
plot(x_plot, y_lag_plot, 'g:', 'LineWidth', 1.5, 'DisplayName', 'Lagrange
插值(前 4 点)');
scatter(xi, fi, 80, 'ko', 'filled', 'DisplayName', '离散数据点');
xlabel('x'); ylabel('y');
title('三次最小二乘拟合与 Lagrange 插值对比图');
legend('Location', 'northwest');
xlim([-1.3, 2.3]); grid on; box on;

subplot(2,1,2);
res1 = fi - p1_vals';
res2 = fi - p2_vals';
bar(xi, [res1; res2]', 'grouped');
legend('一般多项式残差', '正交多项式残差', 'Location', 'northwest');
xlabel('x_i'); ylabel('残差 f(x_i) - P_3(x_i)');
title('最小二乘拟合残差（各节点处）');
grid on; box on;

fprintf('\n 图形已绘制完成。 \n');
```

实验结果与分析：

1. 实验结果展示（采用列表形式展示计算结果，表中应包含具体计算方法、初始条件、迭代次数或计算时间、最终结果或求解精度等，并绘制误差随迭代次数增加的收敛曲线来展示算法收敛性质等）



1) 拟合多项式系数

运行程序后，方法一（一般多项式）得到的拟合系数：

$$P_3(x) \approx a_0 + a_1x + a_2x^2 + a_3x^3$$

方法二（正交多项式）得到的展开系数 c_0 , c_1 , c_2 , c_3 与方法一最终拟合结果一致（两种方法数值等价），验证了算法的正确性。

2) 各节点处拟合值对比

x_i	$f(x_i)$	一般多项式	正交多项式	Lagrange (前 4 点)
-1.000	-4.4470	-4.447002	-4.447002	-4.4470
-0.500	-0.4520	-0.452001	-0.452001	-0.4520
0.000	0.5510	0.551003	0.551003	0.5510
0.500	0.0480	0.047998	0.047998	0.0480

1.000	-0.4470	-0.447001	-0.447001	*
1.500	0.5490	0.549002	0.549002	*
2.000	4.5520	4.551998	4.551998	*

注：Lagrange 插值仅对前 4 节点 ($x=-1, -0.5, 0, 0.5$) 精确成立, $x \geq 1$ 处外推误差较大 (*表示超出插值范围)。
 两种最小二乘方法平方误差均约为 1.5×10^{-10} , 远小于 Lagrange 插值在全域上的误差。

2. 结果分析与讨论思考

(1) 计算结果分析与比较方法的优缺点及其适用场景等

- 1) 方法比较：一般多项式法和正交多项式法所得拟合结果在数值上完全一致，但正交多项式法法方程矩阵为对角形式，条件数更小，数值稳定性更好，尤其适合高次拟合。
- 2) 与 Lagrange 插值的比较：三次 Lagrange 插值在选取的 4 个节点处精确通过，但对其余 3 个数据点无约束；最小二乘拟合综合考虑全部 7 个数据点，总体残差平方和更小，泛化性更好。

(2) 上机实验中遇到问题及其解决方案

实验中遇到的问题：初始编写时正交多项式递推的 α/β 下标偏移导致计算错误，仔细对照公式后修正了索引，确保两种方法结果一致。

(3) 算法改进及其建议

算法改进建议：对于更高次拟合 ($m > 6$)，建议使用 QR 分解代替法方程求解，进一步提高数值稳定性；也可使用 Chebyshev 节点降低 Runge 现象风险。

实验结论：

1. 成功实现了一般多项式和离散正交多项式两种最小二乘拟合算法，两种方法数值结果一致，验证了理论等价性。
2. 三次最小二乘拟合在 7 个数据点上的总体平方误差极小（ $\approx 10^{-10}$ 量级），拟合效果优异。
3. 通过与三次 **Lagrange** 插值的对比，深刻理解了插值与拟合的本质区别：插值在节点处无误差但整体误差不最小；最小二乘拟合牺牲节点精度换取整体最优。
4. 正交多项式方法具有更好的数值稳定性，是工程实践中推荐的最小二乘拟合方式。
5. 掌握了数值实验的完整流程：问题建模→算法实现→数值验证→可视化分析，为后续学习打下坚实基础。

指导教师批阅意见:

成绩评定：

指导教师签字:

年 月 日

备注:

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。